INTERNSHIP REPORT

Of

A Web Development Internship at Thiranex: Building Responsive, Interactive, and Full-Stack JavaScript Applications

(Personal Portfolio, To-Do List, Weather Dashboard & ShopSphere Capstone Project)

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE

By

Roll Number of Student: 2410031072

Name of Student: VEDANSH JAISWAL

Batch: 2024–2028

2026-27

CANDIDATE'S DECLARATION

I, Vedansh Jaiswal, do hereby declare that the internship report titled "A Web Development Internship at Thiranex: Building Responsive, Interactive, and Full-Stack JavaScript Applications" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The work described in this report is based on the projects and tasks undertaken by me during my internship at Thiranex. All references and resources used in the preparation of this report have been appropriately acknowledged.

I further affirm that this report is my own work, has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Signature
Student Name: Vedansh Jaiswal
Roll No.: 2410031072

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during my internship at Thiranex.

I am thankful to the Thiranex team and mentors for providing me with practical, project-based learning opportunities in web development. The internship allowed me to work on different areas of JavaScript and modern web development, including DOM manipulation, client-side state management, browser storage, asynchronous programming, REST API integration, responsive design, and application deployment.

I would also like to thank the School of Computer Science and Engineering, IILM University, and the faculty members involved in coordinating the internship for their guidance and support.

Finally, I am grateful to my parents and everyone who encouraged and supported me throughout the internship.

Signature

Student Name: Vedansh Jaiswal
Roll No.: 2410031072

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 3: INTERNSHIP COMPLETION CERTIFICATE

3.1 Offer Letter

Thiranex
SKILL DEVELOPMENT & FUTURE TECH

Date: 31 July 2026

TO,
Vedansh Jaiswal
Intern ID: THX-JUL3126-333

Subject: Offer Letter for Internship in Web Development

Dear **Vedansh Jaiswal,**

Following our selection process, we are pleased to offer you an internship position at **Thiranex**. We believe your skills and enthusiasm will be a great addition to our team.

Internship Role	**Intern - Web Development**
Commencement Date	**31 Jul 2026**
Completion Date	**30 Aug 2026**
Work Mode	**Remote / Project-Based**

During this tenure, you will work on practical projects under industry mentorship. Your progress will be reviewed periodically, and your performance will determine the successful completion of the internship.

We look forward to a mutually beneficial learning journey. Please confirm your acceptance by starting your onboarding tasks.

Thiranex

This is an electronically generated document. No physical signature is required.
For verification, contact Thiranex Verification Cell.
www.thiranex.in

MSME

Figure 3.1: Thiranex Internship Offer Letter

Offer Letter Details:

- Organization: Thiranex
- Intern: Vedansh Jaiswal
- Intern ID: THX-JUL3126-333

- Role: Intern – Web Development
- Commencement Date: 31 July 2026
- Completion Date: 30 August 2026
- Work Mode: Remote / Project-Based

The offer letter states that the internship involved practical projects under industry mentorship, with periodic progress review.

3.2 Certificate of Achievement

Figure 3.2: Thiranex Certificate of Achievement

The Certificate of Achievement (ID: THX-JUL3126-333), signed by Hariharan M, Founder & CEO of Thiranex, certifies that Vedansh Jaiswal successfully completed an internship in Web Development from 31 July 2026 to 30 August 2026.

CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

This report presents my experience as an Intern – Web Development at Thiranex during the internship period from 31 July 2026 to 30 August 2026.

The internship was conducted remotely on a project-based model. The practical assignments progressively covered semantic HTML5 and accessible, responsive CSS3 design; client-side JavaScript programming and state management; asynchronous programming and REST API integration; and a final web-development capstone involving application architecture, routing, optimization, and deployment.

The practical nature of these tasks provided an opportunity to apply programming concepts to working web applications and to understand how individual web-development concepts fit together in a complete project workflow.

4.2 Organization Profile

Thiranex is the organization where I completed my Web Development internship. The internship was structured as a remote, project-based learning experience.

The official offer letter identifies the position as Intern – Web Development and specifies the internship period from 31 July 2026 to 30 August 2026.

The project-based structure provided practical exposure to different aspects of web development rather than limiting the internship to theoretical learning.

4.3 Internship Problem Statement

A common challenge for students learning web development is moving from individual programming concepts to functional applications.

Understanding HTML5 semantics, CSS3 layout systems, JavaScript syntax, DOM APIs, asynchronous functions, APIs, and deployment in isolation does not necessarily provide experience in combining these concepts into usable applications.

The internship addressed this gap through practical tasks covering:

- Semantic and accessible page structure
- Responsive, mobile-first layout design
- Interactive client-side applications
- CRUD functionality
- State management
- Browser data persistence
- Asynchronous JavaScript
- REST API consumption
- JSON data processing
- Error handling
- Client-side routing
- Application architecture
- Asset optimization
- Deployment

4.4 Project Objectives

The major objectives of the internship were:

- Build a multi-page personal portfolio website using semantic HTML5 markup and accessibility best practices.
- Implement ARIA labels, roles, and SEO-friendly meta tags to achieve strong Lighthouse Accessibility and SEO scores.
- Transform the portfolio into a fully responsive design using advanced CSS3 (Grid, Flexbox, media queries, custom properties).
- Implement a dynamic light/dark theme using CSS custom properties.
- Strengthen practical JavaScript programming skills.
- Understand DOM manipulation and event handling.
- Implement CRUD functionality in a client-side application.
- Understand browser-based state persistence using localStorage.
- Work with asynchronous JavaScript using fetch() and async/await.

- Consume data from a public RESTful weather API and handle nested JSON responses.
- Implement robust error handling for network and API failures.
- Understand modular, component-based frontend architecture using React.
- Implement client-side routing for seamless multi-page navigation.
- Understand basic frontend asset optimization and production builds.
- Deploy a web application to a public hosting platform.

4.5 Scope of the Internship*Table 4.1: Scope of the Internship*

Project	Main Area / Key Concepts
Personal Portfolio Website	HTML5 Semantic Structure & Accessibility / Advanced CSS3 & Responsive Architecture — semantic markup, ARIA, SEO, CSS Grid, Flexbox, light/dark theming
To-Do List	JavaScript Logic & State Management — CRUD, DOM, delegated events, filtering, localStorage
Weather Dashboard	Asynchronous JavaScript & RESTful APIs — fetch, async/await, JSON, OpenWeatherMap API, error handling
ShopSphere	Full-Stack Deployment & Project Architecture — modular React architecture, routing, optimization, deployment

The first project establishes a semantic, accessible, and responsive portfolio using HTML5 and CSS3. The second and third projects focus on client-side JavaScript, browser state, and external API consumption. The final capstone combines these web-development concepts into a deployable, production-oriented application.

Technology / Concept	Purpose
HTML5 (Semantic Elements)	Web page structure (header, nav, main, section, article, footer)
ARIA Labels & Roles	Accessibility for screen readers (WCAG)
CSS3	Styling and layouts
CSS Grid	Two-dimensional page layout
Flexbox	Component-level alignment

4.6 Technologies and Tools Used*Table 4.2: Technologies and Tools Used*

4.7 PROJECT 1 — PERSONAL PORTFOLIO WEBSITE

(HTML5 Semantic Structure & Accessibility — Advanced CSS3 & Responsive Architecture)

4.7.1 Project Overview

The first project was a multi-page personal portfolio website, built across two connected tasks. The first task focused on structuring the site with semantic HTML5 markup and accessibility best practices; the second focused on transforming that structure into a visually polished, fully responsive design using advanced CSS3 techniques. Both tasks were implemented on the same portfolio project and deployed under a single live URL.

4.7.2 Objective — Phase 1: HTML5 Semantic Structure & Accessibility

The objective of this phase was to build a multi-page personal portfolio website adhering strictly to modern semantic HTML5 standards and WCAG accessibility guidelines.

4.7.3 Main Features — Phase 1

- Use of semantic HTML5 tags (header, nav, main, section, article, footer) in place of generic divs
- Implementation of ARIA labels and roles for screen readers
- Creation of optimized, SEO-friendly meta tags
- An accessible, tab-navigable contact form

Expected Outcome: A fully accessible, semantic skeleton of a portfolio website that scores 100 on Lighthouse Accessibility and SEO audits.

4.7.4 Objective — Phase 2: Advanced CSS3 & Responsive Architecture

The objective of this phase was to transform the semantic portfolio into a visually stunning, fully responsive design using advanced CSS3 properties and modern layout architectures.

4.7.5 Main Features — Phase 2

- Use of CSS Grid for complex, two-dimensional page layouts
- Use of Flexbox for localized component alignment

- Mobile-first responsive media queries
- Custom CSS variables for a dynamic light/dark mode theme

Expected Outcome: A pixel-perfect, highly responsive website that adapts flawlessly across all device screen sizes (mobile, tablet, desktop).

Live URL: https://vedanshportfolioo.netlify.app/

4.7.6 Learning Outcome

This project strengthened understanding of:

- Semantic HTML5 structure
- Web accessibility (ARIA, WCAG)
- SEO fundamentals
- CSS Grid and Flexbox layout systems
- Mobile-first responsive design
- CSS custom properties and theming

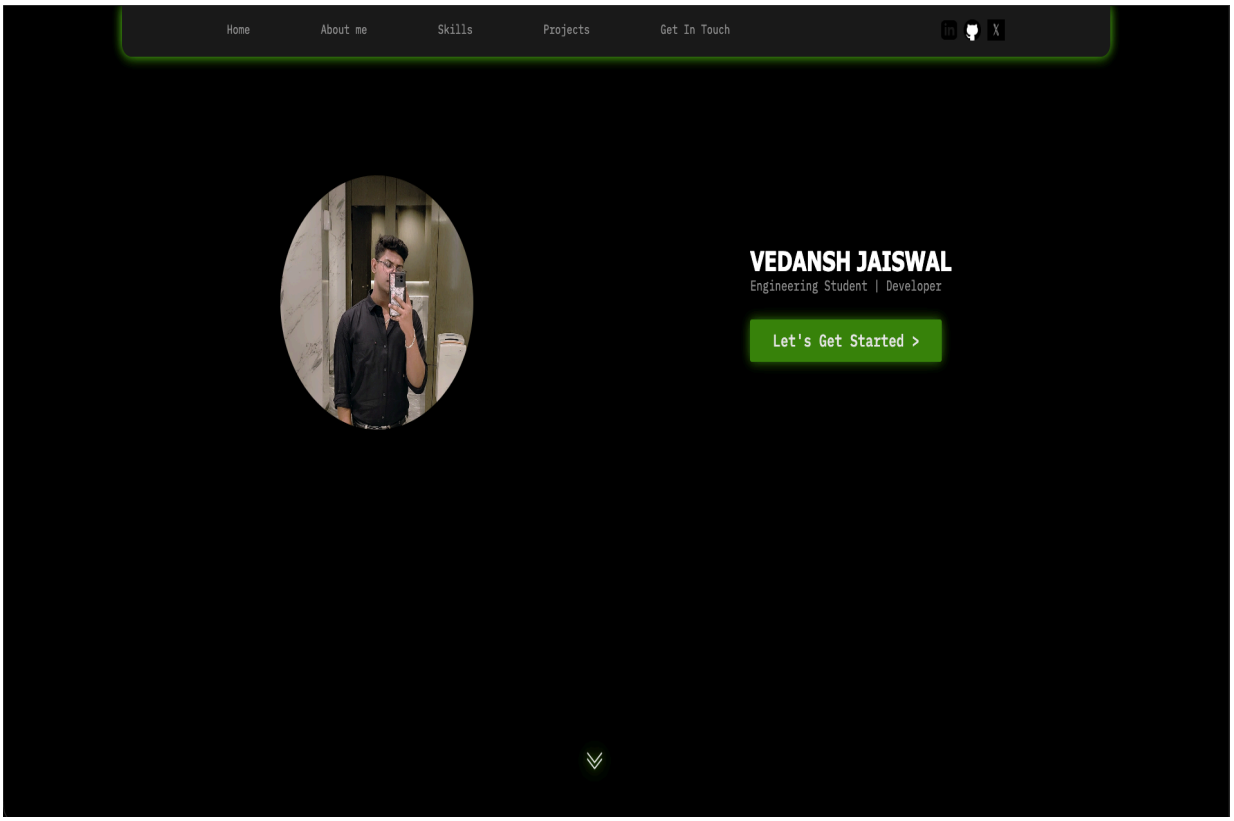


Figure 4.1: Personal Portfolio — home page

4.8 PROJECT 2 — JAVASCRIPT LOGIC & STATE MANAGEMENT

(To-Do List Application)

4.8.1 Project Overview

The second project was an interactive To-Do List application designed to develop practical skills in JavaScript logic and state management.

The task focused on DOM manipulation, event handling, CRUD functionality, filtering, and persistent browser storage.

4.8.2 Objective

The objective was to develop an interactive, client-side To-Do List application to master DOM manipulation, event handling, and local data persistence.

4.8.3 Main Features

- Full CRUD (Create, Read, Update, Delete) functionality
- Automatic data persistence using window.localStorage
- Advanced filtering (All, Active, Completed)
- Dynamically created DOM elements with delegated event listeners
- Live count of remaining tasks

Expected Outcome: A fully functional, state-driven task management application that retains user data across browser reloads.

4.8.4 Application Flow

```
User Action
 ↓
Update JavaScript State
 ↓
Save Data to localStorage
 ↓
Render / Update DOM
 ↓
Display Updated Task List
```

4.8.5 CRUD

Create: Validate the title, create a task object, add it to state, save it, and render it.

Read: Load tasks from state and dynamically display them.

Update: Modify an existing task and persist the updated state.

Delete: Remove the selected task and persist the remaining collection.

4.8.6 Local Storage

The application uses browser storage to retain task information. For example:

```
localStorage.setItem("todo_tasks", JSON.stringify(tasks));
```

On startup:

```
const tasks = JSON.parse(localStorage.getItem("todo_tasks")) ||
[];
```

This allows tasks to remain available after a browser refresh.

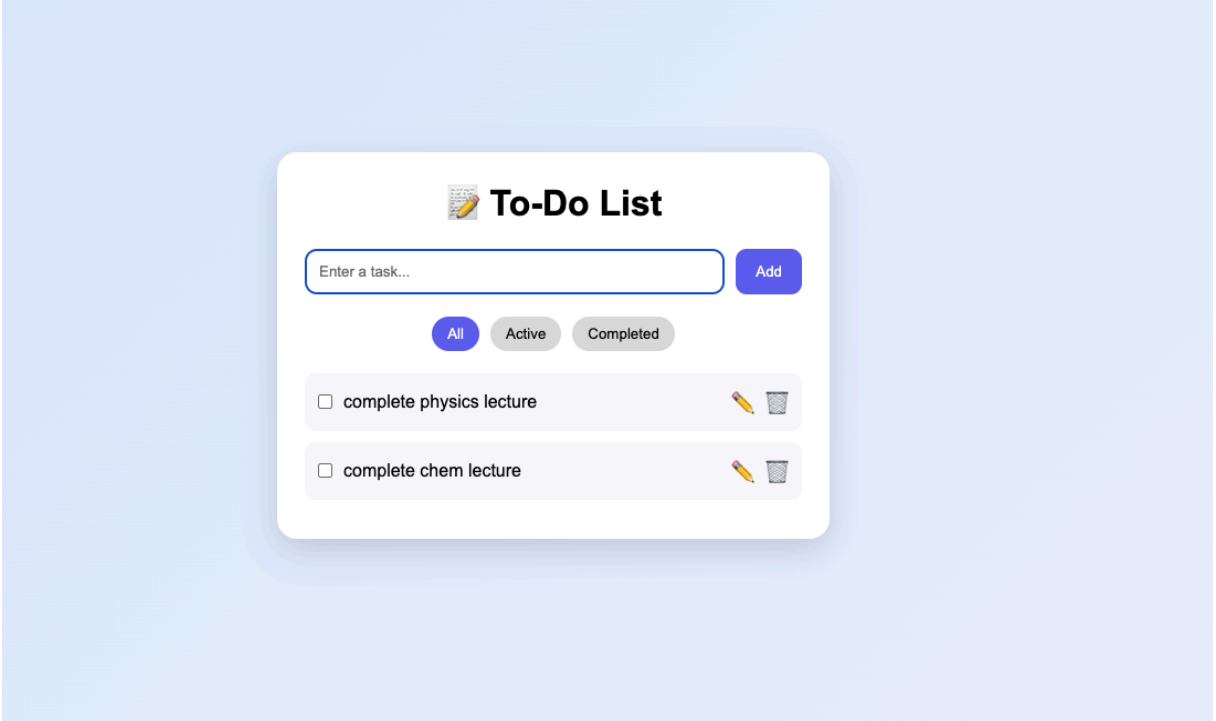
4.8.7 Learning Outcome

This project strengthened understanding of:

- JavaScript arrays and objects
- CRUD operations
- DOM manipulation
- Event handling (including delegated events)
- State-driven UI updates
- JSON serialization
- Browser storage
- Filtering logic

Project Files:

https://drive.google.com/drive/folders/1QCj-h0WEtM6kcHb_l-zyy_e-Dwul9tX_?usp=drive_link


To-Do List
Enter a task...
Add
All
Active
Completed
complete physics lecture
complete chem lecture

4.9 PROJECT 3 — ASYNCHRONOUS JAVASCRIPT & RESTFUL APIS

(Real-Time Weather Dashboard)

4.9.1 Project Overview

The third project was a real-time Weather Dashboard, a dynamic web application that allows users to search for a city and view its current weather information. The task focused on asynchronous JavaScript, API requests, JSON processing, and error handling.

4.9.2 Objective

The objective was to build a real-time Weather Dashboard by fetching and processing JSON data from a public REST API using asynchronous JavaScript.

4.9.3 Main Features

- Real-time data retrieval using the modern Fetch API and async/await
- Comprehensive error handling for failed network requests, invalid cities, and invalid API keys
- Parsing and dynamic rendering of complex, nested JSON objects
- Search functionality to retrieve weather by city name

Expected Outcome: A dynamic dashboard that displays accurate, live weather metrics (temperature, humidity, wind speed) based on user input.

The dashboard displays current temperature, humidity, wind speed, atmospheric pressure, and weather conditions for the searched city, retrieved from the OpenWeatherMap REST API.

4.9.4 Application Flow

```
User enters city
 ↓
Validate input
 ↓
Send REST API request (OpenWeatherMap)
 ↓
Receive JSON response
 ↓
```

```
Parse required values
 ↓
Update DOM
 ↓
Display weather information
```

4.9.5 Asynchronous Programming

The dashboard retrieves live weather data using the Fetch API together with async/await, ensuring the request is handled in a clean, readable, and non-blocking manner:

```
async function getWeather(city) {
  const url = `https://api.openweathermap.org/data/2.5/weather`
+
    `?q=${city}&units=metric&appid=${API_KEY}`;
  const response = await fetch(url);
  if (!response.ok) throw new Error("City not found");
  const data = await response.json();
  return data;
}
```

API Provider: OpenWeatherMap (openweathermap.org).

4.9.6 Error Handling

The application handles cases such as empty city input, an invalid city name, incorrect or missing API keys, network failure, and general API-related errors. A try/catch structure prevents application crashes and provides clear feedback to the user.

4.9.7 Loading State

A loading indicator is displayed while the asynchronous request is being processed.

4.9.8 Learning Outcome

This project provided practical understanding of:

- Promises
- fetch()
- async/await
- REST APIs
- JSON parsing of nested objects

- HTTP response handling
- Error handling
- Dynamic data rendering

Project Files:

https://drive.google.com/drive/folders/151uQUSzZK9sd3iyyM3cY8oNcYlzx0E3b?usp=drive_link

Figure 4.3: Weather Dashboard — city weather lookup

4.10 PROJECT 4 — FULL-STACK DEPLOYMENT & PROJECT ARCHITECTURE CAPSTONE

(ShopSphere — E-Commerce Product Catalog)

4.10.1 Project Overview

The final project was ShopSphere, a Web Development Capstone built as a responsive e-commerce product catalog using React. The task specification emphasized modular architecture, client-side routing, asset optimization, and live deployment.

4.10.2 Objective

The objective was to combine all prior knowledge to build, optimize, and deploy a comprehensive Web Development Capstone Project — an e-commerce product catalog.

4.10.3 Main Features

- Modular frontend application built with reusable React components
- Client-side routing for seamless navigation without full page reloads
- Product browsing, search, and category filtering
- Product details, shopping cart, and wishlist pages
- Product information retrieved dynamically through a REST API
- localStorage used to persist cart and wishlist data across sessions
- Responsive layouts, loading states, error handling, and optimized image loading
- Optimized assets (images, minified code) for maximum performance

Expected Outcome: A live, production-ready web application with a public URL that demonstrates professional-grade web development capabilities.

4.10.4 Technologies Used

- HTML5 — application structure and semantic content
- CSS3 / Tailwind CSS — responsive styling and modern UI design
- JavaScript / TypeScript — application logic, interactions, API communication, and state management

- React — modular, component-based frontend architecture
- Client-Side Routing — seamless navigation between pages
- REST API — dynamic retrieval of product information
- localStorage — persistence of cart and wishlist data
- Git & GitHub — version control and project management
- Netlify / Vercel / Lovable Hosting — deployment and public access

4.10.5 Application Architecture

The application follows a modular architecture using reusable React components, with dedicated components for product cards, product grids, the shopping cart, and the wishlist, alongside separate pages for the catalog, product details, cart, and user-related views. Product data retrieval and business logic are separated into dedicated service modules, keeping the UI layer and the data layer decoupled.

4.10.6 Client-Side Routing

Client-side routing allows SPA-style navigation between the product catalog, individual product detail pages, the cart, the wishlist, and informational pages without full-page reloads, giving the application a fast, native-app-like feel.

4.10.7 Asset Optimization and Deployment

The capstone emphasized frontend performance practices, including a production build with minified code, appropriately sized images, and responsive image handling. The application was deployed to a modern hosting platform to obtain a live, publicly accessible URL, fulfilling the capstone's deployment requirement.

4.10.8 Learning Outcome

The capstone provided experience in application architecture, component/module organization, client-side routing, state management with React, production builds, asset optimization, responsive application development, and deploying an application to a live production environment.

Live URL: https://shopsphere-capstone.lovable.app/

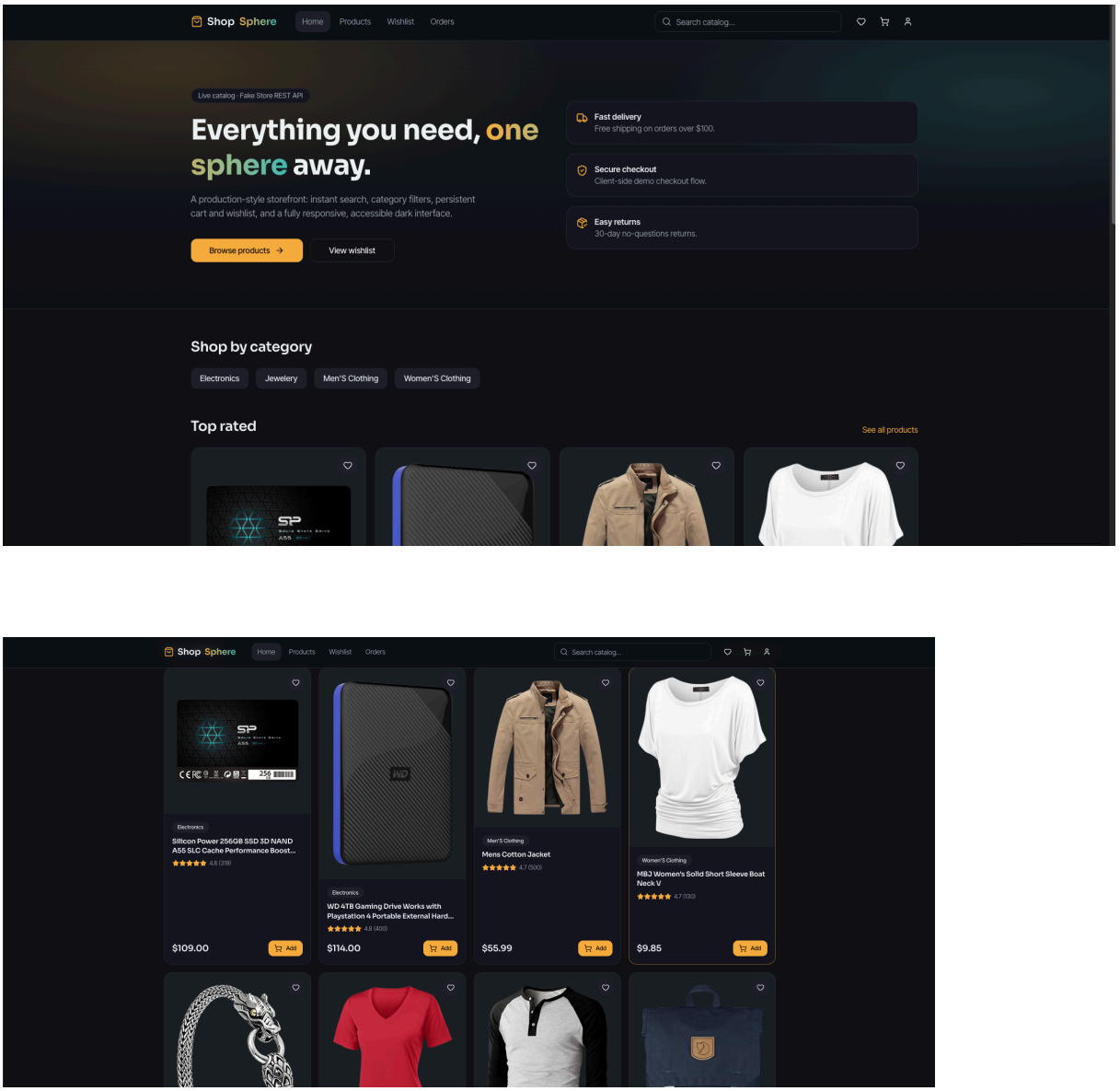


Figure 4.4: ShopSphere — product catalog

4.11 Development Methodology

A practical, iterative development approach was followed across all four projects:

Step 1 — Requirement Analysis

The requirements and expected outcomes of each task were reviewed before implementation.

Step 2 — Planning

The required UI, state, functions, data flow, and project structure were considered.

Step 3 — Core Implementation

Required functionality was implemented before optional improvements.

Step 4 — Error Handling and Validation

Input validation, API failures, empty states, and common edge cases were considered.

Step 5 — Testing

Applications were tested through normal user flows and common failure scenarios.

Step 6 — Finalization and Deployment

Responsive styling, production optimization, and deployment requirements were addressed where applicable.

4.12 Learning Outcomes

The internship strengthened practical understanding across the following areas:

HTML5 & Accessibility

- Semantic markup
- ARIA roles and labels
- SEO-friendly structure

- Accessible forms

CSS3 & Responsive Design

- CSS Grid
- Flexbox
- Media queries
- Custom properties / theming

JavaScript

- Variables and data structures
- Functions
- Arrays and objects
- Array methods
- Conditional logic
- Event handling
- DOM manipulation

State Management

- Maintaining application state
- Updating state after user actions
- Rendering UI from state
- Persisting state in browser storage

Web APIs

- HTTP requests
- REST APIs
- fetch()
- JSON responses
- async/await
- Error handling

Frontend Development

- Responsive layouts
- Reusable UI structures / components
- Client-side navigation
- Dynamic rendering
- Loading and error states

Deployment

- Production builds
- Asset optimization
- Hosting
- Public URLs
- Production verification

4.13 Challenges and Solutions

Challenge 1 — Achieving Full Accessibility and SEO Scores

Reaching a perfect Lighthouse Accessibility and SEO score required more than visually correct markup. Solution: semantic HTML5 tags were used consistently, ARIA labels and roles were added where native semantics were insufficient, and meta tags were reviewed and optimized for SEO.

Challenge 2 — Building a Pixel-Perfect Responsive Layout

Ensuring the portfolio looked polished across mobile, tablet, and desktop screens required careful planning. Solution: CSS Grid was used for page-level layout, Flexbox for component alignment, and mobile-first media queries were used to progressively enhance the design for larger screens.

Challenge 3 — Synchronizing UI and State

When task data changes, the displayed interface must reflect the latest state. Solution: task information was maintained in a central JavaScript data structure, and the relevant UI was updated after each state change.

Challenge 4 — Browser Data Persistence

JavaScript variables disappear after a browser refresh. Solution: localStorage was used with JSON serialization to save and restore task, cart, and wishlist data.

Challenge 5 — Asynchronous API Requests and API Key Handling

External API requests can take time, fail, or return errors due to invalid API keys or invalid city names. Solution: async/await, try/catch, HTTP response checks, and dedicated loading/error states were used to keep the dashboard reliable and user-friendly.

Challenge 6 — Modular Application Structure

Larger applications become difficult to maintain when all functionality is placed in one file. Solution: components, pages, services, and application logic were separated into logical modules in the ShopSphere React application.

Challenge 7 — Deployment

An application that works locally can still encounter issues in production. Solution: a production build was created, and navigation, assets, data loading, and responsiveness were verified after deployment to a live hosting platform.

4.14 Expected / Final Outcomes

The internship tasks were designed to provide practical experience in web development. The outcomes were:

- A semantic, accessible, and responsive personal portfolio website scoring highly on Lighthouse Accessibility and SEO audits.
- A functional JavaScript To-Do application demonstrating CRUD and browser persistence.
- A dynamic Weather Dashboard demonstrating asynchronous API communication and robust error handling.
- A live, production-ready e-commerce capstone (ShopSphere) demonstrating architecture, routing, optimization, and deployment.
- Improved understanding of client-side and full-stack application development.

- Practical experience with debugging and handling common application issues.
- Familiarity with preparing and deploying a web application for production.

4.15 Certificates and Communication Proof

The following evidence supports the completion of this internship:

- Internship offer letter (Chapter 3, Figure 3.1)
- Internship completion certificate (Chapter 3, Figure 3.2)

Source files and live deployments for the four applications are retained and available as supporting proof:

- Personal Portfolio Website (Live) — https://vedanshportfolioo.netlify.app/
- To-Do List (JavaScript Logic & State Management) — https://drive.google.com/drive/folders/1QCj-h0WEtM6kcHb_l-zyy_e-Dwul9tX_?usp=drive_link
- Weather Dashboard (Asynchronous JavaScript & RESTful APIs) — https://drive.google.com/drive/folders/151uQUSzZK9sd3iyyM3cY8oNcYlzx0E3b?usp=drive_link
- ShopSphere (Full-Stack Deployment & Project Architecture Capstone) —
- https://shopsphere-capstone.lovable.app/

CHAPTER 5: CONCLUSION

The Web Development internship at Thiranex provided practical exposure to the process of developing interactive, responsive, and full-stack web applications.

The internship progressed from semantic, accessible HTML5 and responsive CSS3 design, through fundamental client-side JavaScript concepts, to asynchronous programming and API integration, followed by a broader capstone focused on application architecture and live deployment.

The Personal Portfolio Website strengthened understanding of semantic markup, accessibility, SEO, and advanced CSS3 responsive design, including CSS Grid, Flexbox, and custom-property-based theming. The To-Do List project strengthened understanding of CRUD operations, DOM manipulation, delegated event handling, state management, and browser persistence. The Weather Dashboard introduced asynchronous JavaScript, REST API communication with OpenWeatherMap, nested JSON processing, and comprehensive error handling. The final capstone, ShopSphere, brought together these web-development concepts through a modular React architecture, client-side routing, state management, and production deployment.

Overall, the internship helped bridge the gap between academic programming concepts and practical, production-oriented web application development. It also provided experience in considering user interaction, application state, error handling, responsiveness, and preparing an application for live deployment.

CHAPTER 6: BIBLIOGRAPHY / REFERENCES

[1] Thiranex, "Internship Offer Letter — Intern – Web Development," 31 July 2026.

[2] Thiranex, "Certificate of Achievement — Web Development Internship," 30 August 2026.

[3] Thiranex, "Internship Task Specifications" — HTML5 Semantic Structure & Accessibility; Advanced CSS3 & Responsive Architecture; JavaScript Logic & State Management; Asynchronous JavaScript & RESTful APIs; Full-Stack Deployment & Project Architecture.

[4] MDN Web Docs, "JavaScript." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[5] MDN Web Docs, "Web Storage API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

[6] MDN Web Docs, "Fetch API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[7] OpenWeatherMap, "Weather API Documentation." [Online]. Available: https://openweathermap.org/api

[8] React, "React Documentation." [Online]. Available: https://react.dev/

[9] Tailwind CSS, "Documentation." [Online]. Available: https://tailwindcss.com/docs

[10] Vedansh Jaiswal, "Personal Portfolio Website (live deployment)." [Online]. Available: https://vedanshportfolioo.netlify.app/

[11] Vedansh Jaiswal, "To-Do List (project files)." [Online]. Available: https://drive.google.com/drive/folders/1QCj-h0WEtM6kcHb_l-zyy_e-Dwul9tX_?usp=drive_link

[12] Vedansh Jaiswal, "Weather Dashboard (project files)." [Online]. Available: https://drive.google.com/drive/folders/151uQUSzZK9sd3iyyM3cY8oNcYlzx0E3b?usp=drive_link

[13] Vedansh Jaiswal, "ShopSphere (live deployment)." [Online]. Available: https://shopsphere-capstone.lovable.app/